

OOP in JavaScript – Study Notes

Sep 25, 2026 · @Palakolanu Rishith Reddy

Object-oriented programming (OOP) organizes code around **objects** — bundles of data and behavior — instead of loose functions and variables. JavaScript supports OOP through classes, introduced in ES6. These notes cover six core ideas: class & object, constructor, encapsulation, inheritance, polymorphism, and abstraction — each with a plain-language explanation, a real-world example, and a runnable code snippet.

1. Class and Object

A **class** is a blueprint for making things. An **object** is a real thing made from that blueprint, created with the `new` keyword. A function written inside a class is called a **method**, and every object made from the class can use it.

💡 Example: "Car" is a class. "My red car" is an object made from it.

```
function classAndObject() {  
  class Car {  
    drive() {  
      return "The car is driving 🚗";  
    }  
  }  
  
  let myCar = new Car(); // create an object  
  show(myCar.drive());  
}
```

2. Constructor

A **constructor** is a special method that runs automatically when an object is created with `new`. It sets starting values for the object. Inside a class, `this` means "the current object", so `this.name = name` saves the value inside that object.

💡 Example: When a student joins college, their name and age are recorded right at the start.

```
function constructorDemo() {  
  class Student {  
    constructor(name, age) {  
      this.name = name;  
      this.age = age;  
    }  
  }  
  
  let s1 = new Student("Ravi", 20);  
  let s2 = new Student("Priya", 21);  
  
  show("Student 1: " + s1.name + ", age " + s1.age);  
  show("Student 2: " + s2.name + ", age " + s2.age);  
}
```

3. Encapsulation

Encapsulation means hiding data inside a class and allowing access only through methods, which keeps the data safe from wrong changes. In JavaScript, a `#` before a variable name makes it **private** — it cannot be used from outside the class.

💡 Example: An ATM. You can't touch the cash directly; you must use the buttons.

```
function encapsulation() {  
  class BankAccount {  
    #balance = 0;    // private variable  
  
    deposit(amount) {  
      this.#balance = this.#balance + amount;  
    }  
  
    getBalance() {  
      return this.#balance;  
    }  
  }  
  
  let acc = new BankAccount();  
  acc.deposit(500);  
  
  show("Balance using method: " + acc.getBalance());  
  show("Direct access acc.balance: " + acc.balance + " (hidden! ✅)");  
}
```

4. Inheritance

Inheritance lets a child class get the properties and methods of a parent class, using the keyword `extends`. Its main benefit is **reusability** — the same code doesn't need to be rewritten in every class.

💡 Example: A dog is an animal, so `Dog` gets everything `Animal` has, plus its own `bark()`.

```
function inheritance() {
  class Animal {                // parent
    eat() {
      return "I can eat 🍖";
    }
  }

  class Dog extends Animal {    // child
    bark() {
      return "I can bark 🐶";
    }
  }

  let d = new Dog();
  show(d.eat());                // from parent
  show(d.bark());               // Dog's own
}
```

5. Polymorphism

Poly means many, **morph** means forms — **polymorphism** means the same method name behaves differently in different classes. When a child class rewrites a parent's method, it's called **method overriding**.

💡 Example: Every animal can `speaks()`, but a cat says Meow and a cow says Moo.

```
function polymorphism() {
  class Animal {
    speak() {
      return "Animal makes a sound";
    }
  }

  class Cat extends Animal {
    speak() {
      return "Cat says Meow 🐱";
    }
  }

  class Cow extends Animal {
    speak() {
      return "Cow says Moo 🐮";
    }
  }
}
```

```

    }
  }

  let animals = [new Animal(), new Cat(), new Cow()];

  for (let a of animals) {
    show(a.speak()); // same call, different result
  }
}

```

6. Abstraction

Abstraction means showing only *what* something does and hiding *how* it does it. The user gets one simple method to call, while the complicated steps stay hidden inside as private methods.



Example: You press "Start" on a washing machine without knowing how the motor works.

```

function abstraction() {
  class WashingMachine {
    #fillWater() { return "Filling water..."; }
    #spinDrum() { return "Spinning drum..."; }
    #drainWater() { return "Draining water..."; }

    start() {
      show(this.#fillWater());
      show(this.#spinDrum());
      show(this.#drainWater());
      show("Clothes are clean! ✨");
    }
  }

  let wm = new WashingMachine();
  wm.start(); // user only calls start()
}

```